

Whisper of the Countryside

Submitted By

Student Name	Student ID
Rifah Sunjidah Remu	232-15-130
Jannatul Ferdaues	232-15-689
Rifah Sunjidah Urmee	232-15-038

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE412:**
Computer Graphics Lab in the Computer Science and
Engineering Department



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

August 10, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Husne Mubarak, Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Husne Mubarak

Lecturer

Department of Computer Science and Engineering

Daffodil International University

Submitted by

Rifah Sunjidah Remu 232-15-130 Dept. of CSE, DIU	
Jannatul Ferdaues 232-15-689 Dept. of CSE, DIU	Rifah Sunjidah Urmee 232-15-038 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and Relate classes, objects, members of the class, and relationships among them needed for solving specific problems
CO2	Formulate knowledge of object-oriented programming and Java in problem solving
CO3	Analyze Unified Modeling Language (UML) models to Present a specific problem
CO4	Develop solutions for real-world complex problems applying OOP concepts while evaluating their effectiveness based on industry standards.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1,EP3
CO2	PO2	C2	KP3	EP1,EP3
CO3	PO3	C4, A1	KP3	EP1,EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1,EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Feasibility Study	2
1.5 Gap Analysis	2
1.6 Project Outcome	2
2 Proposed Methodology/Architecture	3
2.1 Requirement Analysis & Design Specification	3
2.1.1 Overview	3
2.1.2 Proposed Methodology/ System Design	3
2.1.3 UI Design	4
2.2 Overall Project Plan	5
3 Implementation and Results	6
3.1 Implementation	6
3.2 Performance Analysis	7
3.3 Results and Discussion	7
4 Engineering Standards and Mapping	8
4.1 Impact on Society, Environment and Sustainability	8
4.1.1 Impact on Life	8
4.1.2 Ethical Aspects	8
4.1.3 Sustainability Plan	8
4.2 Project Management and Team Work	9
4.3 Complex Engineering Problem	9
4.3.1 Mapping of Program Outcome	9
4.3.2 Complex Problem Solving	9
4.3.3 Engineering Activities	10

5 Conclusion	11
5.1 Summary	11
5.2 Limitation	11
5.3 Future Work	12
References	12

Chapter 1

Introduction

This chapter introduces the interactive rural village graphics project, explains the motivation for selecting the problem, defines the objectives, discusses technical feasibility and identifies the gap addressed by the implementation.

1.1 Introduction

The project is an interactive 2D rural village scenery application implemented in C using OpenGL and GLUT. Instead of relying on a static image, the scene is procedurally constructed from points, lines, polygons, circles, ellipses and other geometric primitives. The environment contains a river, houses, fields, trees, animals, people, vehicles and other village elements. Dynamic states such as day, night and rain make the scene interactive and animated.

1.2 Motivation

The main motivation is to demonstrate how fundamental Computer Graphics concepts can be integrated into a single working application. A rural environment provides many opportunities to apply different primitives and transformations: circles for wheels and small circular details, ellipses for organic shapes, polygons for buildings and terrain, and transformations for moving and rotating objects. The project also provides a practical way to understand how animation state, user input and rendering are coordinated in an event-driven graphics program.

1.3 Objectives

- Construct a complete rural village scene using OpenGL primitives.
- Implement DDA, Midpoint Circle and Bresenham rasterization algorithms.
- Apply translation, scaling and rotation to scene objects.
- Create continuous animation for boats, cars, fish, birds, people, clouds, water and other objects.
- Implement automatic and manual day/night transitions using elapsed time.
- Implement a 200-particle rainfall effect and night-time fireflies/stars.
- Provide keyboard interaction for object control, rain and day/night states.
- Add audio feedback for the river ambience and vehicle horn.

1.4 Feasibility Study

The implementation is technically feasible because it relies on standard OpenGL/GLUT functionality and basic C programming constructs. The scene is built procedurally, so no large image dataset is required. The animation state is represented with simple scalar variables and small fixed-size arrays such as the 200 rain particles and 15 fireflies. The main platform dependency is the current Windows Multimedia API implementation used for WAV audio.

1.5 Gap Analysis

A simple graphics assignment may demonstrate one algorithm or one static object. This project combines several syllabus-level graphics techniques inside one interactive environment. The main gap addressed is the integration of rasterization, transformations, procedural modeling, animation, environmental state changes, user interaction and sound into a single demonstrable scene.

1.6 Project Outcome

The completed application demonstrates a functional animated village scene with multiple interacting visual states. It provides direct evidence of rasterization algorithms, geometric modeling, transformations, animation, keyboard input and multimedia integration in a Computer Graphics context.

Chapter 2

Proposed Methodology/Architecture

This chapter describes the system requirements, design approach and overall execution flow used to construct the interactive graphics application.

2.1 Requirement Analysis & Design Specification

The system requires a C compiler with OpenGL/GLUT support and, for the audio features, a Windows environment supporting the Multimedia API calls used by the program. The graphics window is initialized at 1200×800 pixels. The application uses an orthographic 2D coordinate system with an approximately -210 to 210 horizontal range and -220 to 310 vertical range.

2.1.1 Overview

The program follows an event-driven rendering architecture. The main function initializes GLUT and the scene, registers the display and keyboard callbacks, and enters the GLUT main loop. The display/update routine draws the scene, updates animation state and requests another redraw. Keyboard input changes state variables such as rain, day/night mode, car position and boat scale.

2.1.2 Proposed Methodology/ System Design

Table 2.1: Simplified system execution flow

Program Start
GLUT Initialization & Window Creation
init(): coordinate system, sounds, particles and state
display(): draw scene using geometric primitives
Update animation variables and environmental states
Keyboard callback changes interactive state
glutPostRedisplay / next frame
Continuous Interactive Scene

2.1.3 UI Design

The user interface is the OpenGL rendering window itself. It presents the village visually and provides keyboard-driven interaction rather than conventional buttons or menus. The main controls are: +/-/r for boat scaling/reset, a/d for car movement, h for horn, s/t for rain start/stop, and D/N for manual day/night selection.

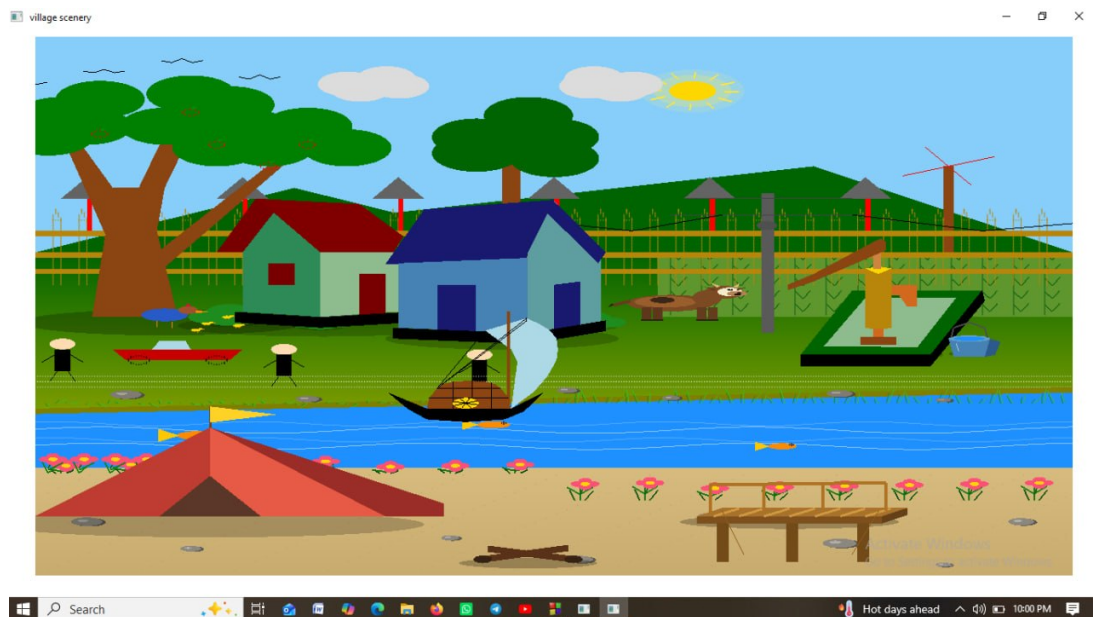


Figure 2.1: Daylight View

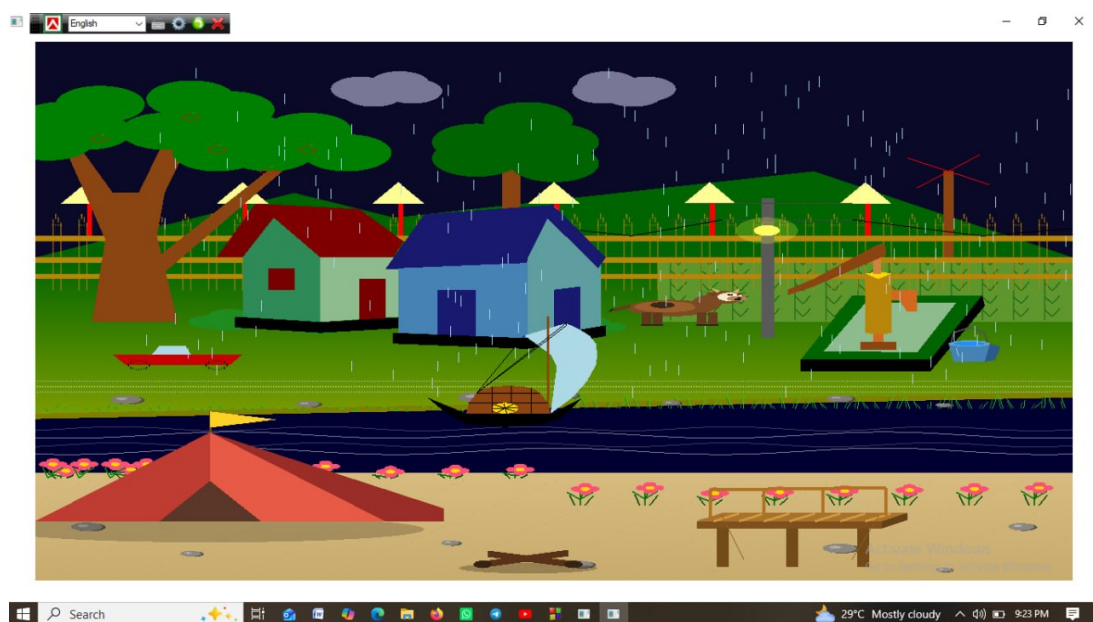


Figure 2.2: Night View

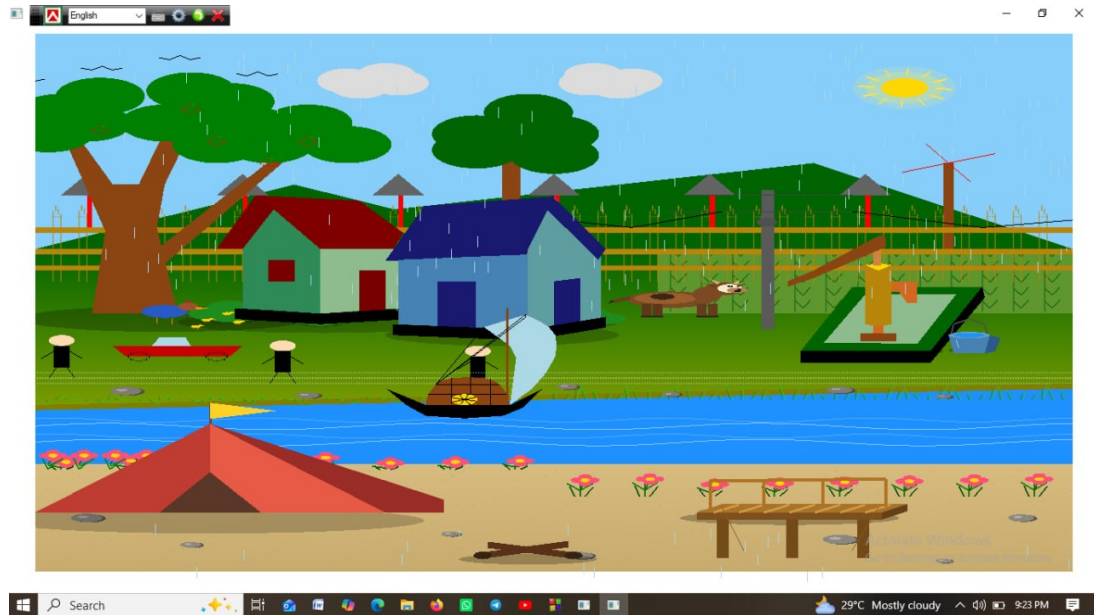


Figure 2.3: Rainy Day

2.2 Overall Project Plan

- Phase 1: Define the village scene and coordinate layout.
- Phase 2: Implement reusable geometric drawing helpers.
- Phase 3: Implement DDA, Midpoint Circle and Bresenham algorithms.
- Phase 4: Build composite objects and environment layers.
- Phase 5: Add transformations and object animation.
- Phase 6: Add day/night, rain, stars, fireflies and water effects.
- Phase 7: Add keyboard controls and audio.
- Phase 8: Test the complete interactive scene and prepare the demonstration.

Chapter 3

Implementation and Results

This chapter explains how the proposed design is implemented in C/OpenGL and discusses the observed behavior of the completed scene.

3.1 Implementation

The implementation is modular. Reusable functions construct objects such as circles, ellipses, fish, cows, cars, bushes, tents, bonfires, humans and environmental elements. The rasterization section explicitly contains the DDA, Midpoint Circle and Bresenham implementations. The display logic then combines these components into the final scene. DDA line rasterization. The DDA function computes dx and dy , chooses the maximum absolute difference as the number of steps, calculates x/y increments and plots successive points. It is used for the walking path. Midpoint Circle. The circle function begins at $(0,r)$, maintains a decision variable $d = 1 - r$, and plots eight symmetric points per iteration. The method is used for circular details including apples, car wheels and a fish eye. Bresenham line. The line routine uses an error term and directional increments to rasterize line segments efficiently. It is used in the fence construction. Transformations. Translation is used for moving objects such as the boat, car, people and clouds. Scaling is used for objects such as the boat and cow. Rotation is used for the windmill/gear and rotating sun rays. Push/pop matrix operations isolate transformations so that one object's transformation does not unintentionally affect other objects. Animation. State variables are continuously changed between frames. The boat position, gear angle, star brightness, fireflies, water offset, people, birds and other moving elements are updated. The day/night system uses GLUT elapsed time and a 10,000 ms phase duration. Weather and environment. Rain is represented by arrays of 200 x/y particle positions. Each frame moves particles downward and recycles particles that leave the scene. Water ripples use a sinusoidal expression with an increasing offset. Fireflies use time-dependent sine/cosine motion and brightness. Audio. The code uses Windows Multimedia/MCI functions to open and play WAV files. The river sound is restarted when the driver reports that playback has stopped, while the horn is triggered through keyboard input.

3.2 Performance Analysis

The implementation uses fixed-size arrays and relatively lightweight geometric primitives, making the scene suitable for an educational real-time application. The rain system processes 200 particles and the firefly system maintains 15 particles. No formal FPS benchmark or timing profile is present in the source code, so performance is assessed qualitatively rather than reported as a measured frame rate.

3.3 Results and Discussion

The final application provides a dynamic village scene rather than a static drawing. The user can observe automatic day/night changes, moving objects, rain, water motion and night effects, while keyboard controls provide direct interaction. The implementation successfully integrates the major graphics concepts targeted by the project.

Chapter 4

Engineering Standards and Mapping

This chapter relates the implemented graphics project to engineering practice, sustainability, teamwork and complex problem-solving considerations.

4.1 Impact on Society, Environment and Sustainability

The software has negligible direct physical environmental impact because the scene is digitally generated. It can be used to demonstrate rural environments without requiring physical models or printed visual materials. The project also provides a foundation for future educational simulations involving weather, agriculture or rural infrastructure.

4.1.1 Impact on Life

The project is primarily educational. It supports learning by turning abstract graphics algorithms into a visible interactive system. Similar visual simulation techniques can also support digital storytelling, educational visualization and interactive demonstrations.

4.1.2 Ethical Aspects

The project does not process personal data or make decisions about individuals. Ethical practice for this type of software includes respecting third-party assets if audio or graphics are introduced later, documenting dependencies, and avoiding misleading claims about physical realism. The current scene is a graphics demonstration rather than a scientific environmental model. The project can be maintained by keeping drawing functions modular, replacing machine-specific file paths with configurable relative asset paths, documenting keyboard controls, and separating update logic from rendering logic. A future version could use reusable structures for object state and double buffering for smoother animation.

4.1.3 Sustainability Plan

The project can be maintained by keeping drawing functions modular, replacing machine-specific file paths with configurable relative asset paths, documenting keyboard controls, and separating update logic from rendering logic. A future version could use reusable structures for object state and double buffering for smoother animation.

4.2 Project Management and Team Work

Provide a cost analysis in terms of budget required and revenue model. In case of budget, you must show an alternate budget and rationales.

4.3 Complex Engineering Problem

The project is structured into manageable technical components: environment construction, reusable object functions, rasterization algorithms, animation, interaction and audio. A practical team workflow is to assign modules, integrate them through shared scene-state variables, test each module independently, and then perform final integration testing. Cost analysis: The implementation primarily requires a development computer and free/open graphics libraries. No paid dataset or cloud service is required by the source code. The main optional cost is obtaining properly licensed audio assets if the project is redistributed. An alternative zero-cost approach is to use self-created or openly licensed WAV files.

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	Applies foundational mathematics and computing concepts through coordinate geometry, trigonometric functions, and rasterization.
PO2	PO2 Designs and implements a working graphics solution by decomposing a complex scene into reusable functions, states and algorithms.
PO3	PO3 Evaluates and integrates multiple interacting requirements including rendering, animation, input, environmental states

4.3.2 Complex Problem Solving

The project demonstrates complex problem-solving by requiring in-depth knowledge of Computer Graphics concepts such as geometric primitives, coordinate systems, rasterization algorithms, transformations, animation, and OpenGL/GLUT programming. The implementation combines multiple graphical components within a single environment, requiring careful analysis of object positioning, rendering order, and animation behavior to ensure smooth and realistic scene generation.

In addition, the project must balance several conflicting requirements, including visual quality, interactivity, animation smoothness, and computational efficiency. The implementation follows OpenGL/GLUT programming standards and API requirements, making it necessary to apply appropriate coding practices while integrating multiple graphical and multimedia features into a unified application.

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Require- ments	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stake- holder Involve- ment	EP7 Inter- dependence
✓	✓	✓		✓		

4.3.3 Engineering Activities

The project utilizes various resources, including a computer system, C programming language, OpenGL/GLUT libraries, development tools, and multimedia resources. These resources are integrated to create an interactive graphics application that demonstrates practical engineering skills and effective utilization of available technologies. The project also involves significant interaction between rendering, animation, keyboard controls, environmental effects, and audio features. Its innovative aspect lies in combining traditional Computer Graphics algorithms such as DDA, Bresenham Line, and Midpoint Circle with animation, day-night transitions, rainfall effects, and user interaction to create a complete and engaging rural village simulation. (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of re- sources	EA2 Level of Interac- tion	EA3 Innovation	EA4 Consequences for society and envi- ronment	EA5 Familiarity
✓	✓	✓		

Chapter 5

Conclusion

This chapter summarizes the completed project, identifies limitations visible in the implementation, and proposes practical future improvements.

5.1 Summary

The project successfully combines fundamental Computer Graphics algorithms and OpenGL/GLUT programming into an interactive rural village scene. The implementation includes DDA line rasterization, Midpoint Circle rasterization and Bresenham line drawing, together with procedural geometric construction and 2D transformations. The scene is animated through continuously changing state variables and includes day/night transitions, rain, water ripples, fish, birds, people, fireflies, stars, a moving boat, a controllable car, a windmill/gear and audio feedback. The project demonstrates that a visually rich graphics application can be constructed from simple primitives when those primitives are organized into reusable functions and coordinated through a central rendering and state-update process.

5.2 Limitation

- The current audio implementation contains machine-specific Windows file paths, reducing portability.
- The application uses single buffering; double buffering would generally provide smoother animation and reduce flicker.
- Most object geometry is hard-coded using coordinate values, which makes large-scale scene editing more time-consuming.
- The source contains some redundant state such as `nightMode` while `isNight` controls the main day/night behavior.
- No formal FPS, CPU or memory benchmark is included in the source, so performance analysis is qualitative.
- The project is primarily a 2D orthographic scene; its depth is visual rather than a full 3D rendering pipeline.

5.3 Future Work

- Replace absolute audio paths with relative/configurable asset paths.
- Use GLUT_DOUBLE and glutSwapBuffers() for smoother animation.
- Separate rendering and update logic into clearer modules.
- Represent objects and animation state with reusable structures/classes.
- Add mouse interaction or an on-screen control panel.
- Add additional weather states such as fog or storms.
- Introduce a more systematic camera/viewport system and optional 3D extensions.
- Add automated testing for keyboard states and animation transitions.

References